

Rapport de projet universitaire

Stabilisation du workflow documentaire de Marmote et intégration de notebooks C++

Projet réalisé du 12 février 2026 au 1 juin 2026

Membre du projet

FERRADJ Assia

Table des matières

1	Introduction	4
2	Contexte du projet	4
3	Présentation des missions et cahier des charges	6
3.1	Stabilisation du workflow et de l'arborescence	6
3.2	Préparation des environnements de travail	7
3.3	Intégration des notebooks C++	7
3.4	Publication et qualité documentaire	7
4	Descriptif détaillé des missions et manière d'appréhender le projet	8
4.1	Stabiliser avant d'étendre	8
4.2	S'appuyer sur deux sources de vérité	8
4.3	Procéder par validations successives	8
4.4	Conserver une logique de livrable final	9
5	Réalisation des missions	9
5.1	Correction des problèmes de structure et de chemins	9
5.2	Sécurisation des scripts	9
5.2.1	Contrôle du contexte d'exécution	9
5.2.2	Mode <code>dry_run</code>	10
5.2.3	Exécution des commandes externes	10
5.3	Création d'un script global de génération	10
5.4	Préparation des notebooks Python et des variantes Colab	10
5.5	Intégration des notebooks C++ avec Xeus-cling	11
5.5.1	Choix de l'environnement	11
5.5.2	Étapes concrètes de création d'un notebook C++	12
5.5.3	Construction des notebooks C++	15
5.5.4	Transcription des exemples et couverture des notebooks	15
5.5.5	Formalisation d'une méthode de génération avec Codex	15
5.6	Résolution des incohérences entre la documentation et le code réel	17
5.6.1	Limites d'une alternative sous Windows : <code>jupyter-cpp-kernel</code>	18
5.7	Stabilisation des kernels et nettoyage des sorties	18
5.8	Export des notebooks C++ vers des fichiers <code>.cpp</code>	18
5.9	Gestion de CMake et compilation des exemples	19
5.10	Publication Sphinx et ajustements documentaires	20
5.10.1	Refonte de l'index <code>cpp_examples.rst</code>	21
5.11	Usage des tags de masquage et cohérence pédagogique	24
5.12	Nettoyage final et livrabilité	24
6	Difficultés rencontrées	25
6.1	Difficultés liées à l'arborescence et aux scripts	25
6.2	Difficultés liées aux environnements	25
6.3	Difficultés liées aux notebooks C++	25
6.4	Difficultés liées à la publication documentaire	25
6.5	Bilan des difficultés	26
7	Conclusion	26

A	Annexes	27
A.1	Installation de Sphinx et des extensions	27
A.2	Génération de la documentation HTML avec Sphinx	27
A.3	Création de l'environnement Xeus-cling	27
A.4	Empilement d'environnements Conda	27
A.5	Vérification des kernels Jupyter	27
A.6	Extrait de redirection automatique vers le bon dossier	27
A.7	Extrait du parsing d'arguments	28
A.8	Extrait de génération des exports C++	28
A.9	Exemple de tags pour masquer une cellule technique	28
A.10	Configuration Marmote dans un notebook C++	28
A.11	Vérification de l'installation de Marmote dans Colab	29

1 Introduction

Dans le cadre de ma Licence MIAGE, j’ai réalisé un projet long autour de la librairie scientifique **Marmote** (*MARKovian MOdelling Tools and Environments*). Marmote est une bibliothèque dédiée à la modélisation et à la résolution de modèles de Markov à variables discrètes. Elle propose une API C++ et une API Python, cette dernière encapsulant les objets C++ afin de simplifier leur utilisation dans des notebooks scientifiques.

Le projet s’inscrit dans un contexte à la fois académique et applicatif. D’un côté, il s’agissait de comprendre une base logicielle déjà existante, utilisée pour produire une documentation scientifique. De l’autre, il fallait faire évoluer cette base sans rompre le workflow de génération déjà en place. L’enjeu n’était donc pas seulement de produire du nouveau contenu, mais surtout de rendre ce contenu cohérent, automatisable et maintenable.

Le site Marmote est généré avec **Sphinx**. Les exemples Python et C++ y sont présentés sous forme de pages HTML, de fichiers sources téléchargeables et d’archives. Avant la génération du site, plusieurs scripts Python préparent les ressources : copie des exemples C++, production des fichiers `.res` et `.cmd`, préparation de notebooks, génération d’archives, mise à jour de fichiers `CMakeLists.txt`, etc. C’est cette chaîne de préparation qui a constitué le cœur du travail. Le dépôt utilisé pendant le projet est disponible à l’adresse suivante : https://github.com/AssFrDz/projet_universitaire.

Ce rapport présente donc à la fois le cadre du projet, les missions qui m’ont été confiées, la manière dont je les ai abordées, les réalisations effectivement menées, puis les principales difficultés rencontrées. L’objectif est de montrer comment un travail de maintenance et d’extension progressive peut aboutir à un workflow de documentation plus robuste, mieux documenté et davantage exploitable.

2 Contexte du projet

Organisation du travail

Le projet a été mené de manière itérative, en lien régulier avec mon enseignant encadrant. Les réunions de suivi permettaient de présenter les tâches réalisées, de faire le point sur les blocages techniques, puis de définir les priorités suivantes. Cette organisation progressive a été particulièrement utile car les difficultés rencontrées étaient souvent très concrètes : erreurs de chemins, dépendances absentes, comportements différents entre Windows et Linux, incohérences entre la documentation et l’implémentation réelle, ou encore instabilité du kernel C++.

Au fil des échanges, le travail a donc pris une double dimension :

- une dimension de **maintenance logicielle**, visant à stabiliser un flot d’exécution existant ;
- une dimension de **production documentaire**, visant à intégrer des notebooks C++ dans un site déjà structuré autour des notebooks Python.

État initial du workflow

Au départ, les ressources Python et C++ étaient préparées par des chemins différents. Les exemples C++ étaient compilés et exécutés afin de produire les sorties

associées, tandis que les notebooks Python étaient préparés séparément pour la documentation et pour Google Colab.

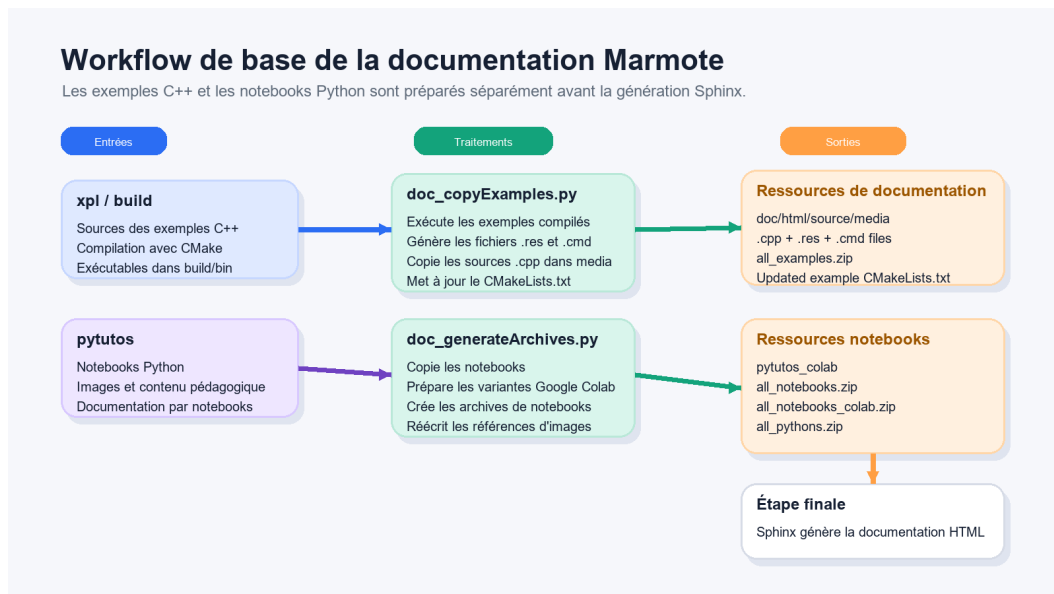


FIGURE 1 – Workflow initial de préparation des ressources de documentation.

Dans ce schéma, les scripts `doc_copyExamples.py` et `doc_generateArchives.py` jouaient déjà un rôle central, mais l'ensemble restait fragmenté. Les notebooks C++ n'étaient pas encore intégrés comme une composante standard de la documentation, et les étapes de génération n'étaient pas assez mutualisées.

Objectif général

L'objectif général du projet était de faire évoluer cette organisation vers un workflow plus unifié, capable de traiter à la fois :

- les exemples C++ compilés ;
- les notebooks Python ;
- les notebooks C++ pour Xeus-cling ;
- les archives téléchargeables de la documentation.

Les objectifs opérationnels peuvent être résumés ainsi :

- corriger et stabiliser les *issues* existantes dans les scripts ;
- fiabiliser les répertoires, chemins et binaires utilisés ;
- intégrer les notebooks C++ dans la documentation ;
- générer des exports C++ cohérents à partir des notebooks ;
- améliorer la lisibilité de la documentation générée ;
- sécuriser les scripts avec des contrôles de contexte et des modes de simulation.

La problématique qui a guidé l'ensemble du travail est donc la suivante :

Comment intégrer des notebooks Jupyter en C++ dans un workflow de documentation scientifique existant, tout en conservant un fonctionnement fiable, automatisable et maintenable ?

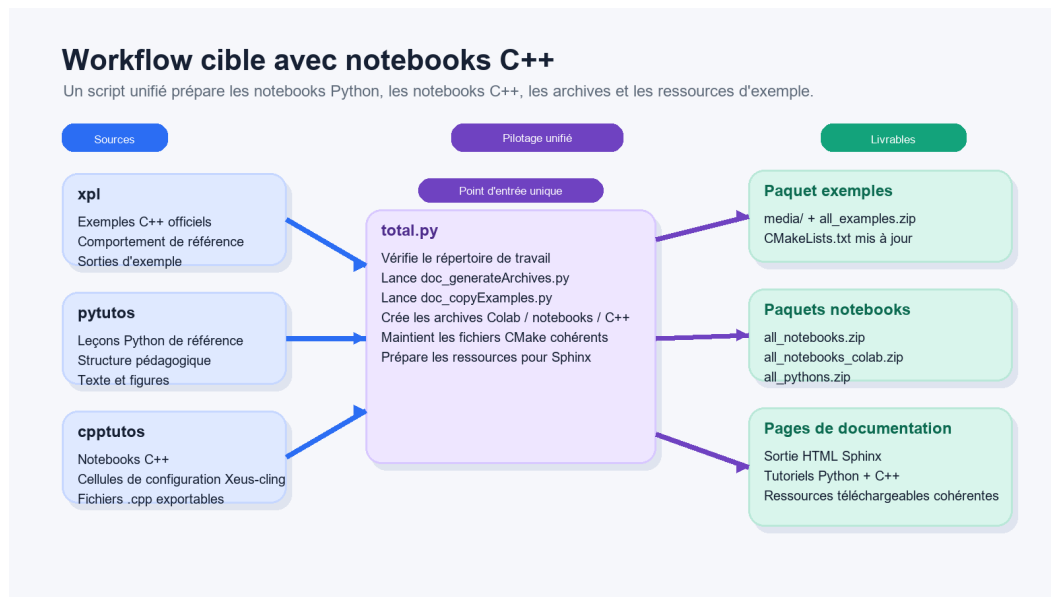


FIGURE 2 – Workflow cible avec intégration des notebooks C++ et d'un script global.

3 Présentation des missions et cahier des charges

Le projet a été structuré autour de quatre grandes missions complémentaires : la stabilisation du workflow, la préparation des environnements, l'intégration des notebooks C++, puis l'amélioration de la publication documentaire. Cette partie joue le rôle d'un cahier des charges synthétique : elle présente, à un niveau encore général, ce que le stage devait permettre d'atteindre.

3.1 Stabilisation du workflow et de l'arborescence

Une première mission a consisté à clarifier l'arborescence du projet et à vérifier que chaque dossier remplissait bien son rôle dans la chaîne de génération.

```
PROJECT-ROOT
  build
  doc
    html
      build
      source
  scripts
  xpl
```

FIGURE 3 – Arborescence principale du projet.

Les rôles principaux sont les suivants :

- **xpl** contient les exemples C++ officiels ;
- **build** contient les binaires compilés et le contexte de compilation ;
- **doc/html/source** contient les sources Sphinx, les notebooks et les ressources ;
- **doc/html/build** contient les pages HTML générées ;
- **scripts** contient les scripts Python qui préparent les ressources avant la génération de la documentation.

Cette phase a mis en évidence plusieurs points faibles :

- des chemins attendus mais absents lors de certains tests ;
- des noms d'exécutables qui ne correspondaient pas aux binaires réellement produits ;
- des incohérences entre les sources présentes dans `xpl`, les fichiers copiés dans `media` et les références utilisées par les scripts.

3.2 Préparation des environnements de travail

Le projet mobilise plusieurs environnements techniques :

- un environnement Python pour Sphinx et les scripts de génération ;
- un environnement Marmote pour exécuter les exemples et préparer les ressources ;
- un environnement Conda dédié à Xeus-cling pour les notebooks C++ ;
- WSL, utilisé pour contourner les limites de Windows sur le support de Xeus-cling.

Cette partie du travail n'était pas seulement une mise en place technique : elle conditionnait aussi la reproductibilité des notebooks et la capacité à reconstruire la documentation de bout en bout.

3.3 Intégration des notebooks C++

L'une des missions centrales a été la création d'une version C++ des notebooks, parallèle à la partie Python déjà présente dans la documentation. L'objectif n'était pas simplement de convertir du code C++ en notebook, mais de proposer un support pédagogique cohérent avec les leçons Python :

- mêmes sections et mêmes titres ;
- mêmes explications textuelles ;
- même ordre de construction des modèles ;
- sorties affichées de manière comparable, ou adaptées lorsque les API diffèrent.

Cette mission a demandé de s'appuyer à la fois sur les notebooks Python (`pytutos/`) et sur les exemples C++ officiels (`xpl/`), avec un travail de réécriture pour résoudre les écarts entre la documentation théorique et l'implémentation réelle.

3.4 Publication et qualité documentaire

Enfin, une dernière mission a porté sur la qualité de la documentation produite :

- génération correcte des pages Sphinx ;
- mise à jour des index C++ ;
- masquage des cellules techniques non pédagogiques ;
- réduction des sorties trop volumineuses dans les notebooks ;
- production d'archives cohérentes pour le téléchargement des notebooks et fichiers C++.

Cette phase a été essentielle pour rendre le dépôt réellement livrable, au sens où les contenus générés sont exploitables à la fois par un lecteur de la documentation et par un développeur souhaitant récupérer les ressources.

4 Descriptif détaillé des missions et manière d’appréhender le projet

Même si les objectifs du projet pouvaient être formulés de manière relativement claire, leur mise en œuvre ne pouvait pas se faire sous la forme d’une refonte immédiate. L’une des premières décisions méthodologiques a donc été de procéder par étapes, en stabilisant d’abord l’existant avant d’ajouter de nouvelles briques.

4.1 Stabiliser avant d’étendre

Le dépôt manipulait déjà plusieurs types de ressources : exemples C++, exécutables, sorties d’exécution, notebooks Python, archives et pages Sphinx. Avant d’ajouter des notebooks C++, il était donc nécessaire de s’assurer que le workflow historique restait compréhensible et fonctionnel. Cela a orienté le travail vers une logique simple :

- vérifier l’arborescence réelle du dépôt ;
- identifier les scripts réellement utilisés ;
- reproduire les erreurs observées ;
- corriger les incohérences de chemins et de destinations ;
- seulement ensuite intégrer les nouvelles fonctionnalités.

Cette approche m’a évité de multiplier trop tôt les couches de complexité. En particulier, elle a permis de distinguer clairement ce qui relevait d’un problème de structure du projet et ce qui relevait réellement de l’intégration de Xeus-cling ou des notebooks C++.

4.2 S’appuyer sur deux sources de vérité

Pour la partie notebook C++, la méthode retenue a consisté à toujours travailler à partir de deux références simultanées :

- les notebooks Python présents dans `pytutos/`, qui portaient la structure pédagogique, les titres de sections et le niveau d’explication attendu ;
- les exemples officiels C++ présents dans `xpl/`, qui constituaient la référence la plus fiable sur les appels d’API réellement disponibles.

Cette double lecture a été indispensable. Les notebooks Python fournissaient l’organisation pédagogique, mais ils ne suffisaient pas pour écrire du code C++ exécutable. Inversement, les exemples C++ étaient corrects sur le plan technique, mais trop bruts pour être intégrés tels quels dans un notebook pédagogique. Toute la difficulté a donc consisté à produire une version intermédiaire, à la fois fidèle, exécutable et lisible.

4.3 Procéder par validations successives

Le projet a aussi été mené selon une logique de validation incrémentale. Chaque ajout technique important a été validé à petite échelle avant d’être réinjecté dans le workflow global. Cela a été le cas pour :

- les tests de redirection vers `doc/html/source` ;
- le mode `dry_run` ;
- le lancement d’un premier notebook C++ vide ;
- le chargement des bibliothèques Marmote dans Xeus-cling ;
- la transcription d’un premier exemple complet ;
- l’export des notebooks vers des fichiers `.cpp` ;

— la régénération des archives et des fichiers `CMakeLists.txt`.

Cette manière de faire a limité les régressions. Elle a également facilité les échanges avec l'encadrant, car chaque séance de suivi pouvait s'appuyer sur un état précis du projet : un test réussi, une erreur reproductible, ou une nouvelle fonctionnalité partiellement stabilisée.

4.4 Conserver une logique de livrable final

Enfin, j'ai abordé le projet non pas comme une simple suite d'expérimentations, mais comme la préparation d'un dépôt livrable. Cela a influencé plusieurs décisions :

- nettoyer les commentaires et docstrings des scripts actifs ;
- distinguer les cellules techniques et pédagogiques dans les notebooks ;
- éviter de diffuser des sorties trop volumineuses ou peu utiles ;
- rendre les archives téléchargeables cohérentes avec le contenu de la documentation ;
- maintenir une nomenclature claire entre `pytutos`, `cpptutos`, `media` et `instructions`.

Autrement dit, la mission n'a pas été pensée uniquement en termes de preuve de concept, mais aussi en termes de maintenance, de transmission et de réutilisabilité. C'est cette perspective qui a guidé la plupart des choix d'organisation.

5 Réalisation des missions

5.1 Correction des problèmes de structure et de chemins

La première étape concrète a consisté à reconstruire un cadre d'exécution fiable. Plusieurs erreurs initiales provenaient d'un écart entre l'arborescence supposée par les scripts et l'arborescence réelle du projet. Certains dossiers attendus n'étaient pas présents lors des premiers tests, tandis que des noms d'exécutables ou de répertoires ne correspondaient pas aux fichiers produits.

Le travail a donc porté sur :

- la vérification des dossiers `xpl`, `build/bin`, `doc/html/source/media` et `instructions` ;
- le réaligement des tableaux de correspondance entre exemples, exécutables et paramètres ;
- la remise en cohérence des dossiers utilisés pour copier, zipper et publier les ressources.

Cette phase a permis de remettre en état le flux de base : compilation des exemples, exécution des binaires, génération des fichiers de sortie, copie vers la documentation et création des archives associées.

5.2 Sécurisation des scripts

5.2.1 Contrôle du contexte d'exécution

L'un des apports les plus utiles du projet a été la sécurisation du contexte d'exécution. Les scripts ne doivent pas être lancés depuis n'importe quel répertoire, car ils manipulent des chemins relatifs, déplacent des fichiers et mettent à jour des archives. Une exécution au mauvais endroit peut donc produire des effets de bord importants.

J'ai ajouté puis consolidé une logique de contrôle du répertoire courant. Si le script est lancé depuis un sous-répertoire du projet, il remonte à la racine puis redirige automatiquement vers `doc/html/source`. Si le contexte est incohérent, le script s'arrête.

5.2.2 Mode `dry_run`

Le mode `dry_run` a été généralisé pour permettre de tester le workflow sans provoquer de copies, suppressions ou déplacements réels. Cette possibilité a joué un rôle important dans le diagnostic des erreurs de chemins, dans la mise au point du script global et dans la vérification des enchaînements d'étapes.

5.2.3 Exécution des commandes externes

La partie la plus sensible de `doc_copyExamples.py` concerne l'exécution des binaires générés. Ce point a été revu pour :

- réaligner les noms d'exécutables réellement produits sur Windows, avec le suffixe `.exe` ;
- éviter les appels à des binaires inexistantes ;
- traiter explicitement le cas particulier de `example4`, où le dossier de données et l'exécutable introduisaient une ambiguïté ;
- sécuriser l'appel de commandes externes via `subprocess.run(..., capture_output=True)`.

5.3 Création d'un script global de génération

À mesure que les scripts se stabilisaient, il est devenu pertinent de disposer d'un point d'entrée unique. Nous avons donc consolidé la logique de génération autour de `total.py`.

Ce script global reprend désormais les étapes principales de génération d'archives et de copie d'exemples. Il mutualise également plusieurs comportements auparavant dispersés :

- le parsing des arguments ;
- la gestion du mode `dry_run` ;
- la redirection vers le bon répertoire ;
- la vérification de l'existence des dossiers attendus ;
- l'appel aux fonctions de préparation des notebooks et des exemples.

Dans la pratique, ce script constitue l'ossature du nouveau workflow : il centralise l'exécution, rend les étapes plus lisibles et évite de relancer séparément plusieurs scripts avec des contextes différents.

Ces ajustements ont eu un impact pratique immédiat. Avant cette sécurisation, il fallait constamment se souvenir du bon répertoire de lancement pour chaque script. Après modification, le script global devient plus tolérant au contexte de départ, tout en restant strict lorsque l'exécution sort de l'arborescence prévue. Cela a considérablement réduit les erreurs de manipulation durant les tests.

5.4 Préparation des notebooks Python et des variantes Colab

Le script `doc_generateArchives.py` était déjà chargé de préparer les notebooks Python, mais il a été enrichi et corrigé pour mieux s'intégrer au nouveau flux.

Les traitements effectués sont les suivants :

- copie et renommage des notebooks ;
- ajout automatique des cellules d'installation pour Colab ;
- insertion de cellules de vérification de l'installation de Marmote ;
- ajout d'un *fallback* sur `sys.path` lorsque nécessaire ;
- remplacement des images locales par des URL adaptées pour Colab ;

```

#Si on n'est pas dans un dossier du projet, on quitte
#Si on se trouve dans source, on poursuit le script
#Sinon, on remonte jusqu'à la racine project-root (à remplacer)
#Puis on redirige vers doc/html/source
if("project-root" in current_directory):
    if(os.path.basename(current_directory)=="source"):
        print("No directory change needed",os.getcwd())
    else:
        print("Unexpected working directory. Redirecting to the correct location...")
        while(os.path.basename(os.getcwd())!="project-root"):
            print("Moving up...",os.getcwd())
            os.chdir("..")

        path = os.path.join("doc","html","source")
        os.chdir(path)
        print("Redirection completed successfully : ",os.getcwd())
else:
    print("Wrong directory. Program stops")
    sys.exit()

```

FIGURE 4 – Extrait de la logique de redirection automatique vers doc/html/source.

```

PS C:\Users\assia\Documents\projet_univ\dossier 2\project-root\build\bin> python ../../scripts/test.py
Not exactly the location we expected. Redirecting to the good dir ...
Going up ... C:\Users\assia\Documents\projet_univ\dossier 2\project-root\build\bin
Going up ... C:\Users\assia\Documents\projet_univ\dossier 2\project-root\build
Redirecting operation exited successfully : C:\Users\assia\Documents\projet_univ\dossier 2\project-root\doc\html\source
PS C:\Users\assia\Documents\projet_univ\dossier 2\project-root\build\bin>

```

FIGURE 5 – Test réussi de la redirection depuis le dossier build/bin.

— génération des archives `all_notebooks.zip`, `all_notebooks_colab.zip` et `all_pythons.zip`

Cette partie a également demandé quelques corrections de destination, notamment pour les répertoires d'archives Python, afin d'éviter des déplacements vers des dossiers inexistants.

Nous avons également revu la cellule de vérification de l'installation dans Colab. L'idée n'était pas seulement d'ajouter une cellule technique supplémentaire, mais de guider l'utilisateur lorsque l'import direct de Marmote échoue. La présence d'une cellule `!conda list marmote`, suivie d'un repli possible via `sys.path`, permet de documenter explicitement le diagnostic et la correction, ce qui est particulièrement utile pour un public étudiant ou pour une démonstration à distance.

5.5 Intégration des notebooks C++ avec Xeus-cling

5.5.1 Choix de l'environnement

La question du support C++ dans Jupyter a constitué un point structurant du projet. En théorie, plusieurs solutions existent, mais pour Marmote, la plus adaptée est Xeus-cling. En revanche, sa mise en œuvre sous Windows reste délicate. Les noyaux et dépendances sont beaucoup plus faciles à obtenir sous Linux via `conda-forge`, d'où le choix de travailler avec WSL pour la partie notebook C++.

Le workflow retenu est donc le suivant :

- création d'un environnement `xeus-cpp-env` ;
- installation de `jupyterlab`, `xeus-cling` et `marmote` ;
- lancement de JupyterLab depuis WSL ;
- utilisation du kernel `xcpp17` ou équivalent.

2. Créer un notebook vide et sélectionner le bon kernel La seconde étape consiste à ouvrir un notebook vierge puis à sélectionner le kernel C++ adapté. Dans notre cas, le choix s'est porté sur C++17, qui correspond aux kernels disponibles dans l'environnement Xeus-cling. Cette étape est simple en apparence, mais elle conditionne toute la suite : si le mauvais kernel est sélectionné, les cellules ne s'exécutent pas dans le bon interpréteur.

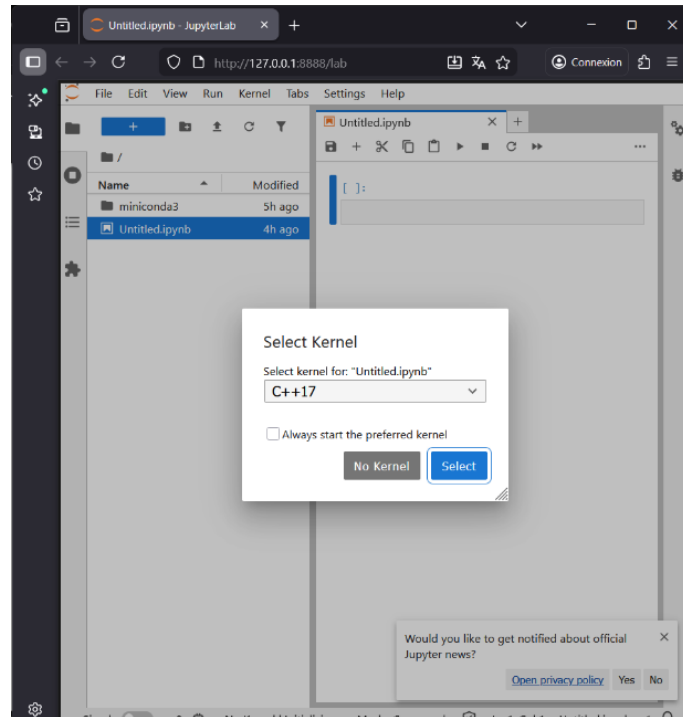


FIGURE 9 – Sélection du kernel C++17 lors de la création d'un notebook.

3. Réaliser un test minimal Avant d'intégrer Marmote, un test minimal est indispensable. Il permet de vérifier que le kernel fonctionne de manière autonome, avant même de charger des en-têtes ou des bibliothèques supplémentaires. Dans notre cas, un simple `std::cout << "ok"` a suffi à valider le fonctionnement du kernel.

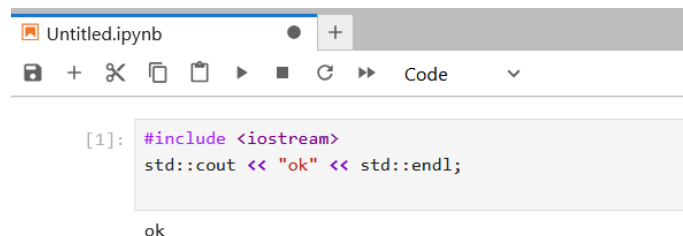


FIGURE 10 – Premier test de fonctionnement du notebook C++ avec une sortie simple.

4. Charger Marmote dans le notebook Une fois ce premier test validé, le notebook peut être enrichi avec la configuration Marmote. C'est à cette étape qu'apparaissent les directives `#pragma cling add_include_path(...)` et `#pragma cling load(...)`. Elles permettent de rendre accessibles les en-têtes et les bibliothèques nécessaires au code C++ Marmote.

```

[1]: #pragma clang add_include_path("/home/assia/miniconda3/envs/xeus-cpp-env/include")
    #pragma clang add_include_path("/home/assia/miniconda3/envs/xeus-cpp-env/include/marmoteCore")
    #pragma clang add_include_path("/home/assia/miniconda3/envs/xeus-cpp-env/include/marmoteMarkovChain")
    #pragma clang add_library_path("/home/assia/miniconda3/envs/xeus-cpp-env/lib")
    #pragma clang load("libmarmoteCore.so")
    #pragma clang load("libmarmoteMarkovChain.so")

[2]: #include <marmoteMarkovChain/marmoteMarkovChain.h>
    #include <marmoteMarkovChain/marmoteSimulationResult.h>
    #include <marmoteCore/marmoteDiscreteDistribution.h>
    #include <marmoteCore/marmoteSparseMatrix.h>
    #include <iostream>
    #include <string>

    unsigned int n = 10;
    double prob1 = 0.1, prob2 = 0.2, prob3 = 0.7;

    double states[3] = {0, 1, 2};
    double* probas = new double[3];
    probas[0] = prob1;
    probas[1] = prob2;
    probas[2] = prob3;

```

FIGURE 11 – Configuration d’un notebook C++ Marmote avec les directives `#pragma clang`.

5. Exécuter un premier exemple Marmote L’étape suivante consiste à reprendre un exemple simple de la documentation C++ et à l’adapter au contexte notebook. Cela implique notamment de retirer la structure `main`, de conserver uniquement le contenu utile des instructions C++, puis d’exécuter le code cellule par cellule. La sortie obtenue confirme que Marmote est bien chargé et que le noyau C++ interprète correctement les objets du modèle.

```

discrete sparse
3
    0      0 2.500000e-01
    0      1 5.000000e-01
    0      2 2.500000e-01
    1      0 4.000000e-01
    1      1 2.000000e-01
    1      2 4.000000e-01
    2      0 4.000000e-01
    2      1 3.000000e-01
    2      2 3.000000e-01

stop
discrete values { 0 1 2 } probas {    0.1    0.2    0.7 }
Trajectory
    0      1 1
    1      2 2
    2      2 2
    3      1 1
    4      0 0
    5      0 0
    6      1 1
    7      0 0
    8      1 1
    9      2 2
   10      2 2

```

FIGURE 12 – Exemple de sortie obtenue après exécution d’un premier notebook Marmote en C++.

Ces différentes étapes ont servi de socle pour l’ensemble des notebooks C++ ensuite produits. Elles ont aussi montré que la réussite du projet ne dépendait pas uniquement du contenu des exemples, mais d’une chaîne complète : environnement, kernel, configuration Cling, chargement de Marmote et structuration pédagogique du notebook.

5.5.3 Construction des notebooks C++

Les notebooks C++ n’ont pas été créés comme de simples copies des exemples `xpl`. Ils ont été reconstruits pour suivre la pédagogie des notebooks Python. Cela a conduit à :

- créer des cellules Markdown explicatives, comme dans les `pytutos` ;
- séparer les cellules techniques des cellules réellement pédagogiques ;
- structurer les leçons en sections comparables aux leçons Python ;
- ajuster les affichages et les sorties pour éviter l’effet “dump technique”.

Dans les notebooks C++, les cellules de configuration Xeus-cling sont désormais placées dans des cellules cachées, via des tags du type `hide-cell` ou `nbsphinx-hidden`, tandis que les `#include` utiles à la compréhension restent visibles lorsque cela a du sens d’un point de vue pédagogique.

5.5.4 Transcription des exemples et couverture des notebooks

Les notebooks C++ générés couvrent désormais les leçons communes entre `pytutos` et `xpl`, ainsi que des exemples supplémentaires nécessaires pour homogénéiser l’ensemble côté C++. Le travail a notamment porté sur :

- les leçons Markov Chain (`Lesson1` à `Lesson7`, puis `Lesson10`) ;
- les leçons MDP (`MDP_Lesson1` à `MDP_Lesson5`) ;
- les leçons d’application (`App_Lesson1` et `App_Lesson2`).

Une attention particulière a été portée à la cohérence entre le nommage des notebooks, les correspondances avec les exemples officiels C++ et les entrées présentes dans `cpp_examples.rst`.

5.5.5 Formalisation d’une méthode de génération avec Codex

Pour homogénéiser la transcription des notebooks, j’ai également formulé un prompt détaillé destiné à un agent Codex. L’idée n’était pas de lui déléguer aveuglément la production des notebooks, mais de formaliser précisément les contraintes à respecter : sources de référence, structure pédagogique, fonctions obsolètes à éviter, style des cellules Markdown, nature des sorties à conserver, etc.

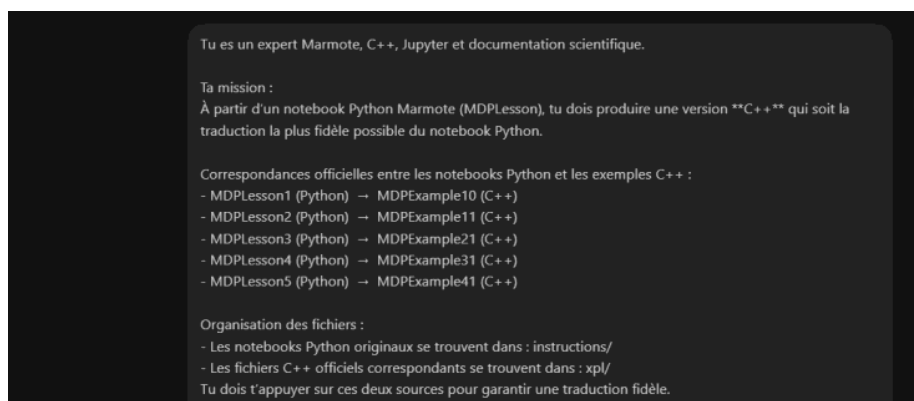


FIGURE 13 – Début du prompt structurant la génération des notebooks C++.

Cette formalisation a été très utile pour deux raisons. Premièrement, elle a permis de systématiser les conversions entre notebooks Python et notebooks C++. Deuxièmement, elle a servi de cadre de vérification : lorsqu’un notebook généré s’écartait de

```
Objectif :  
Créer un notebook C++ qui soit **strictement parallèle** au notebook Python correspondant :  
- mêmes sections  
- mêmes titres  
- mêmes explications textuelles  
- mêmes étapes de construction du modèle  
- mêmes affichages  
- mêmes résultats (autant que possible)  
  
Structure attendue du notebook C++ :  
- Cellule Markdown : titre du notebook  
- Cellule Markdown : introduction  
- Cellule C++ : includes Marmote + initialisations  
- Cellule Markdown : "Définition du modèle"  
- Cellule C++ : création des états, actions, matrices, distributions  
- Cellule Markdown : "Résolution du MDP"  
- Cellule C++ : appel aux méthodes de résolution (PolicyIteration, ValueIteration, etc.)  
- Cellule Markdown : "Analyse des résultats"  
- Cellule C++ : affichage des politiques, valeurs, trajectoires
```

FIGURE 14 – Extrait du prompt décrivant la structure attendue du notebook C++.

la structure attendue ou utilisait une API obsolète, il devenait plus simple d'identifier précisément le défaut et de le corriger.


```

Contraintes de fidélité :
1. Tous les commentaires doivent être pédagogiques, comme dans les notebooks Python.
2. Le style doit être identique aux notebooks Python :
  - phrases explicatives avant chaque bloc
  - commentaires dans le code
  - affichages lisibles
3. Si une fonction Python n'existe pas en C++, tu dois :
  - proposer l'équivalent C++
  - expliquer brièvement la différence
  - adapter le code pour obtenir le même résultat logique.

Contraintes Marmote :
4. Ne pas utiliser les fonctions obsolètes :
  - solpol (n'existe plus)
  - writeSolution() (bug)
5. Utiliser writeSolutionByDim#() si nécessaire.
6. Toujours vérifier la cohérence avec les fichiers C++ officiels dans xpl/.

Entrée :
Je te fournis ci-dessous le contenu du notebook Python Marmote (copié depuis instructions/).

Sortie attendue :
Un notebook complet, alternant cellules Markdown et cellules C++, qui traduit fidèlement le notebook
Python en C++, prêt à être intégré dans la documentation.

```

FIGURE 15 – Extrait du prompt décrivant les contraintes de fidélité et les contraintes Marmote.

5.6 Résolution des incohérences entre la documentation et le code réel

L'intégration des notebooks C++ a mis en évidence plusieurs décalages entre la documentation Marmote et l'implémentation réellement utilisable dans le kernel C++.

Parmi les cas les plus marquants :

- l'ancienne fonction `solpol` n'existe plus ;
- `writeSolution()` ne fonctionnait pas dans notre contexte, alors que des variantes `writeSolutionByDim#()` ou des accès directs à la solution étaient nécessaires ;
- certaines surcharges de fonctions devenaient ambiguës dans Cling ;
- certains opérateurs d'affichage supposés disponibles dans les exemples ne l'étaient pas réellement dans les notebooks.

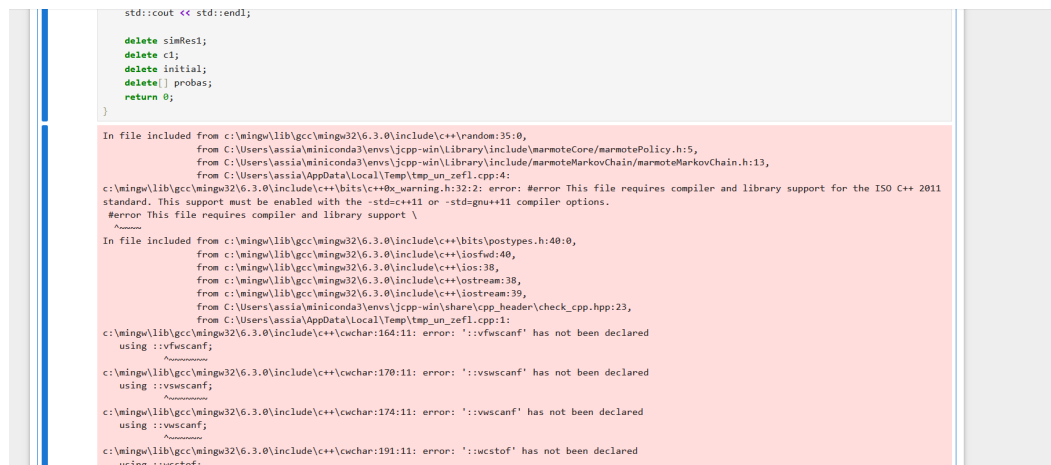
Cela a conduit à une série de corrections ciblées :

- renommage de variables ambiguës comme `beta` pour éviter les conflits avec la bibliothèque standard ;
- désambiguïsation de fonctions comme `SimulateHittingTime` ;
- qualification explicite de symboles liés à `marmote::stateWritePolicy()` ;
- remplacement de certains affichages par `toString(...)` ou par des appels plus robustes ;
- adaptation des démonstrations lorsque certaines fonctions, comme `Embed()`, n'étaient pas disponibles sur les cas infinis.

Ces corrections n'ont pas seulement permis de “faire marcher” les notebooks : elles ont aussi mis en évidence la nécessité d'écrire une documentation fidèle au comportement réel de l'API.

5.6.1 Limites d’une alternative sous Windows : jupyter-cpp-kernel

Avant de retenir définitivement Xeus-cling, une alternative Windows a été testée : `jupyter-cpp-kernel`, basé sur `g++`. Cette piste semblait intéressante, car elle évitait le détour par WSL. En pratique, elle s’est révélée peu adaptée à Marmote.



```
std::cout << std::endl;

delete simRes1;
delete c1;
delete initial;
delete[] probas;
return 0;
}

In file included from c:\mingw\lib\gcc\mingw32\6.3.0\include\c++\random:35:0,
from C:\Users\assia\miniconda3\envs\jcpp-win\library\include\marmoteCore\marmotePolicy.h:5,
from C:\Users\assia\miniconda3\envs\jcpp-win\library\include\marmoteMarkovChain\marmoteMarkovChain.h:13,
from C:\Users\assia\AppData\Local\Temp\tmp_un_zefl.cpp:4:
c:\mingw\lib\gcc\mingw32\6.3.0\include\c++\bits\c++0x_warning.h:32:2: error: #error This file requires compiler and library support for the ISO C++ 2011
standard. This support must be enabled with the -std=c++11 or -std=gnu++11 compiler options.
#error This file requires compiler and library support \
^~~~~~
In file included from c:\mingw\lib\gcc\mingw32\6.3.0\include\c++\bits\postypes.h:40:0,
from c:\mingw\lib\gcc\mingw32\6.3.0\include\c++\iosfwd:40,
from c:\mingw\lib\gcc\mingw32\6.3.0\include\c++\ios:38,
from c:\mingw\lib\gcc\mingw32\6.3.0\include\c++\iostream:38,
from c:\mingw\lib\gcc\mingw32\6.3.0\include\c++\istream:39,
from C:\Users\assia\miniconda3\envs\jcpp-win\share\cpp_header\check_cpp.hpp:23,
from C:\Users\assia\AppData\Local\Temp\tmp_un_zefl.cpp:1:
c:\mingw\lib\gcc\mingw32\6.3.0\include\c++\cuchar:164:11: error: '::vfscanf' has not been declared
using ::vfscanf;
^~~~~~
c:\mingw\lib\gcc\mingw32\6.3.0\include\c++\cuchar:170:11: error: '::vscanf' has not been declared
using ::vscanf;
^~~~~~
c:\mingw\lib\gcc\mingw32\6.3.0\include\c++\cuchar:174:11: error: '::vscanf' has not been declared
using ::vscanf;
^~~~~~
c:\mingw\lib\gcc\mingw32\6.3.0\include\c++\cuchar:191:11: error: '::wcstof' has not been declared
using ::wcstof;
```

FIGURE 16 – Exemple d’erreurs rencontrées avec `jupyter-cpp-kernel` sous Windows.

Les erreurs observées portaient sur le support incomplet de C++11 et sur des incompatibilités de toolchain, notamment via `mingw`. Même après plusieurs ajustements, cette solution restait trop fragile pour charger correctement les en-têtes et dépendances Marmote. Ce test a donc servi à justifier plus clairement le choix de WSL et de Xeus-cling pour le reste du projet.

5.7 Stabilisation des kernels et nettoyage des sorties

Les notebooks C++ se sont révélés plus sensibles que les notebooks Python, notamment parce qu’une cellule réexécutée hors ordre peut laisser des pointeurs invalides en mémoire. Plusieurs noyaux mouraient à cause de ce type de situation, en particulier dans des notebooks manipulant des structures de données ou des objets Marmote libérés explicitement.

Pour réduire ce risque, nous avons :

- réécrit certaines cellules de nettoyage pour remettre les pointeurs à `nullptr` ;
- remplacé certaines lambdas par de vraies fonctions plus stables dans Xeus-cling ;
- ajouté des contrôles défensifs avant certains appels ;
- adapté plusieurs notebooks ([Lesson6](#), [Lesson7](#), [Lesson5](#), etc.) pour éviter des comportements trop fragiles.

Dans le même esprit, les sorties affichées ont été revues. Les notebooks Python de référence ne montrent pas systématiquement tous les dumps ou toutes les traces d’exécution. Nous avons donc aligné les notebooks C++ sur cette logique : seules les sorties réellement pédagogiques sont conservées visibles, tandis que les sorties massives ou secondaires sont cachées.

5.8 Export des notebooks C++ vers des fichiers `.cpp`

Un point important du projet a été la génération de fichiers C++ téléchargeables à partir des notebooks. Cette responsabilité est désormais portée par la fonction `creating_cppFiles(...)`

Au départ, cette étape reposait sur une concaténation de cellules de code, trop fragile pour être fiable.

La fonction `creating_cppFiles(...)` a donc été retravaillée pour s'appuyer sur une logique plus propre :

- utilisation de `nbconvert.ScriptExporter` pour exporter les notebooks ;
- exclusion réelle des cellules techniques marquées `hide-cell` ou `nbsphinx-hidden` ;
- retrait des directives spécifiques à Xeus-cling comme `#pragma cling` ;
- conservation des `#include` utiles dans le fichier C++ final ;
- génération automatique d'un `CMakeLists.txt` associé aux fichiers exportés ;
- ajout de ce `CMakeLists.txt` dans l'archive ZIP des notebooks C++.

Ce point est important : le fichier exporté n'est plus simplement un collage des cellules du notebook, mais une version nettoyée, portable et plus exploitable comme vrai code C++.

5.9 Gestion de CMake et compilation des exemples

Le fichier `CMakeLists.txt` a constitué un autre sujet important. D'une part, il fallait régénérer correctement le `CMakeLists.txt` des exemples à partir d'un modèle. D'autre part, il fallait comprendre pourquoi certains tests de compilation échouaient sous Windows.

Le premier point a conduit à :

- corriger la reconstruction du `CMakeLists.txt` à partir du modèle ;
- s'assurer que l'archive `all_examples.zip` contenait bien la bonne version du fichier ;
- générer également un `CMakeLists.txt` cohérent pour les fichiers `.cpp` exportés depuis les notebooks.

Le second point a nécessité une clarification importante. Lors d'un premier essai, l'erreur renvoyée par `find_package(marmote)` a laissé penser que Marmote ne proposait pas réellement de support CMake sous Windows. Après de nouveaux tests, cette interprétation s'est révélée inexacte. Le support existe bien, et les fichiers de configuration CMake sont présents ; le problème venait surtout du fait que CMake ne savait pas où les chercher. En pratique, il fallait donc indiquer explicitement à Windows la source ou le préfixe d'installation de Marmote, par exemple via `CMAKE_PREFIX_PATH` ou une variable du type `marmote_DIR`. Une fois ce chemin correctement fourni, la configuration fonctionne sans patch spécifique de la bibliothèque.

Il a donc fallu distinguer clairement :

- la compilation des exemples officiels dans `build` ;
- la livraison d'un `CMakeLists.txt` utilitaire dans les archives de notebooks C++.

Ce passage a été déterminant, car il a permis de distinguer deux besoins différents : d'une part, pouvoir continuer à compiler correctement les exemples officiels pour produire les `.res` et `.cmd` ; d'autre part, fournir dans les archives des notebooks C++ un fichier `CMakeLists.txt` utile comme base de réutilisation. Il a aussi permis de reformuler correctement l'erreur observée : il ne s'agissait pas d'une absence de support CMake côté Marmote, mais d'un problème de localisation de cette configuration dans l'environnement Windows utilisé.

```

CMake Error at CMakeLists.txt:23 (find_package):
  By not providing "Findmarmote.cmake" in CMAKE_MODULE_PATH this project has
  asked CMake to find a package configuration file provided by "marmote", but
  CMake did not find one.

Could not find a package configuration file provided by "marmote" with any
of the following names:

  marmoteConfig.cmake
  marmote-config.cmake

Add the installation prefix of "marmote" to CMAKE_PREFIX_PATH or set
"marmote_DIR" to a directory containing one of the above files.  If
"marmote" provides a separate development package or SDK, be sure it has
been installed.

-- Configuring incomplete, errors occurred!

```

FIGURE 17 – Erreur initiale rencontrée lorsque CMake ne connaît pas l'emplacement de la configuration Marmote.

```

(base) C:\Users\assia\Documents\projet_univ\dossier 2\project-root\build>cmake -G "Visual Studio 18 2026" ../xpl
-- Selecting Windows SDK version 10.0.26100.0 to target Windows 10.0.26200.
-- The C compiler identification is MSVC 19.50.35723.0
-- The CXX compiler identification is MSVC 19.50.35723.0
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Check for working C compiler: C:/Program Files (x86)/Microsoft Visual Studio/18/BuildTools/VC/Tools/MSVC/14.50.35717/bin/Hostx64/x64/cl.exe - skipped
-- Detecting C compile features
-- Detecting C compile features - done
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info - done
-- Check for working CXX compiler: C:/Program Files (x86)/Microsoft Visual Studio/18/BuildTools/VC/Tools/MSVC/14.50.35717/bin/Hostx64/x64/cl.exe - skipped
-- Detecting CXX compile features
-- Detecting CXX compile features - done
-- Configuring done (6.6s)
-- Generating done (0.3s)
-- Build files have been written to: C:/Users/assia/Documents/projet_univ/dossier 2/project-root/build

```

FIGURE 18 – Configuration CMake réussie après indication correcte de la source et du contexte de build.

5.10 Publication Sphinx et ajustements documentaires

La génération de la documentation finale avec Sphinx a nécessité plusieurs ajustements.

Parmi les problèmes rencontrés :

- l'extension `recommonmark` manquante ;
- l'absence de `pandoc` pour le rendu des notebooks ;
- l'absence du kernel `xcpp17` dans certains environnements de build ;
- des liens vers des fichiers de téléchargement absents, comme `manual.pdf` ;
- des écarts entre les anciens index C++ et le nouveau dossier `cpptutos`.

Le travail documentaire a donc consisté à :

- mettre à jour les fichiers `cpp_examples.rst` et les références vers `cpptutos` ;
- harmoniser les intitulés des leçons C++ avec ceux des leçons Python ;
- distinguer clairement les cellules techniques cachées et les cellules visibles dans les notebooks ;
- documenter la bonne commande de build Sphinx et le bon environnement Conda à utiliser.

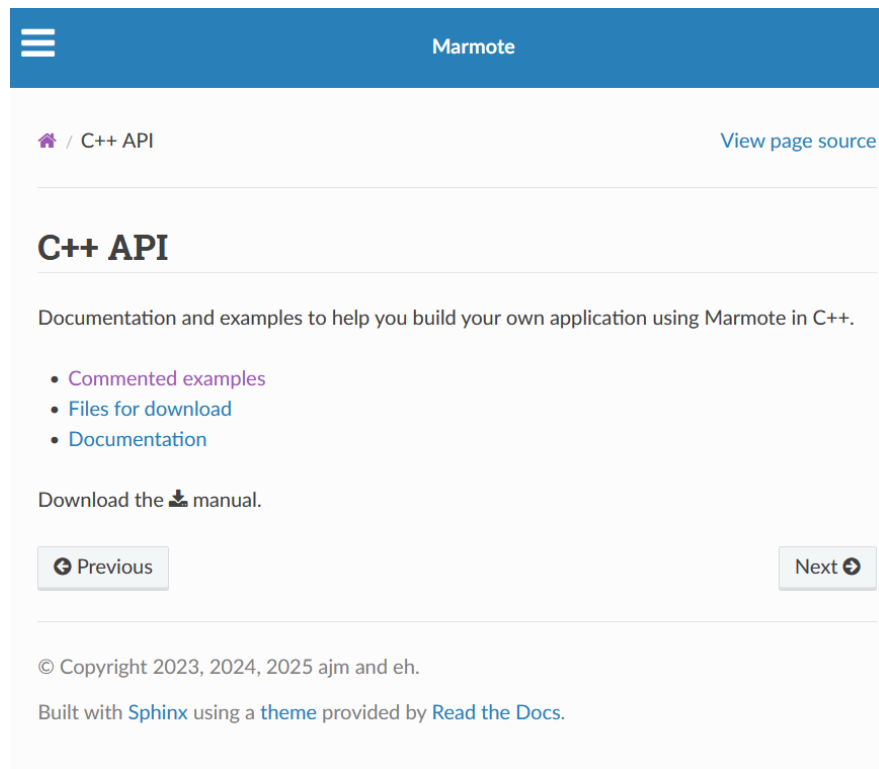


FIGURE 19 – Capture de la page C++ API générée dans la documentation Marmote.

5.10.1 Refonte de l'index `cpp_examples.rst`

La migration du dossier `example/` vers le dossier `cpptutos/` a demandé une mise à jour explicite de l'index C++. L'ancien fichier se basait sur des pages de type `example/exampleX`, alors que la nouvelle version devait refléter le nommage pédagogique des notebooks C++.

Cette évolution ne relève pas d'un simple changement cosmétique. Elle marque le passage d'une logique centrée sur des fichiers d'exemple bruts à une logique de leçons structurées, plus proche de la partie Python et plus cohérente avec l'objectif pédagogique du site.

```

C++ API
=====

.. toctree::
    :hidden:
    :maxdepth: 2
    :caption: Topics
    :name: cpp_toc

    Commented examples <./cpp_examples>
    Files for download <./cpp_downloads>
    Documentation <./cpp_index>

Documentation and examples to help you build your own ap
using Marmote in C++.

- `Commented examples <./cpp_examples.html>`_
- `Files for download <./cpp_downloads.html>`_
- :doc:`Documentation <./cpp_index>`

Download the :download:`manual <./media/manual.pdf>`.

```

FIGURE 20 – Extrait de la source `cpp_api.rst` utilisée pour construire la page C++ API.

```

C++ Tutorial and Examples
-----

.. toctree::
    :maxdepth: 2
    :caption: Table of Examples
    :name: cppexampletoc
    :hidden:

    example/example1
    example/example2
    example/example3
    example/example4
    example/example5
    example/example6
    example/example7
    example/example10
    example/exampleMDP10
    example/exampleMDP11
    example/exampleMDP21
    example/exampleMDP31
    example/exampleMDP40

The following examples show elementary manipulations of Marmote objects:

* :doc:`Example 1 <./example/example1>`: create a Markov Chain and perform Monte-Carlo simulation
* :doc:`Example 2 <./example/example2>`: create a Markov Chain and compute transient distributions
* :doc:`Example 3 <./example/example3>`: as in Example 2 with several chains
* :doc:`Example 4 <./example/example4>`: read and write Markov Chains from/to files
* :doc:`Example 5 <./example/example5>`: create a Markov Chain and compute the stationary distribution with different methods
* :doc:`Example 6 <./example/example6>`: manipulate Sets (state spaces)
* :doc:`Example 7 <./example/example7>`: create Markov Chains and perform structural analysis

```

FIGURE 21 – Ancienne rédaction de l'index C++ basée sur le dossier `example/`.

```
C++ Tutorial and Examples
-----

.. toctree::
   :maxdepth: 2
   :caption: Table of Examples
   :name: cppexampletoc
   :hidden:

   cpptutos/Lesson1.ipynb
   cpptutos/Lesson2.ipynb
   cpptutos/Lesson3.ipynb
   cpptutos/Lesson4.ipynb
   cpptutos/Lesson5.ipynb
   cpptutos/Lesson6.ipynb
   cpptutos/Lesson7.ipynb
   cpptutos/MDP_Lesson1.ipynb
   cpptutos/MDP_Lesson2.ipynb
   cpptutos/MDP_Lesson3.ipynb
   cpptutos/MDP_Lesson4.ipynb
   cpptutos/MDP_Lesson5.ipynb

**Markov Chain Lessons**

1. `Lesson 1: <./cpptutos/Lesson1.html>`_ making Markov chains
2. `Lesson 2: <./cpptutos/Lesson2.html>`_ solving Markov chains
3. `Lesson 3: <./cpptutos/Lesson3.html>`_ working with state spaces
4. `Lesson 4: <./cpptutos/Lesson4.html>`_ working with distributions
5. `Lesson 5: <./cpptutos/Lesson5.html>`_ predefined Markov chains
6. `Lesson 6: <./cpptutos/Lesson6.html>`_ manipulating sets and state spaces
7. `Lesson 7: <./cpptutos/Lesson7.html>`_ structural analysis of Markov chains
8. `Lesson 10: <./cpptutos/Lesson10.html>`_ multidimensional random walks on a large grid
```

FIGURE 22 – Nouvelle rédaction de l’index C++ basée sur cpptutos/.

5.11 Usage des tags de masquage et cohérence pédagogique

La question du masquage des cellules techniques a également constitué un sujet de fond. Dans l'idéal, les cellules de configuration Xeus-cling ne doivent pas polluer la lecture de la documentation finale. Cependant, le comportement des tags dépend à la fois de Jupyter, de `nbsphinx` et du mode de rendu.

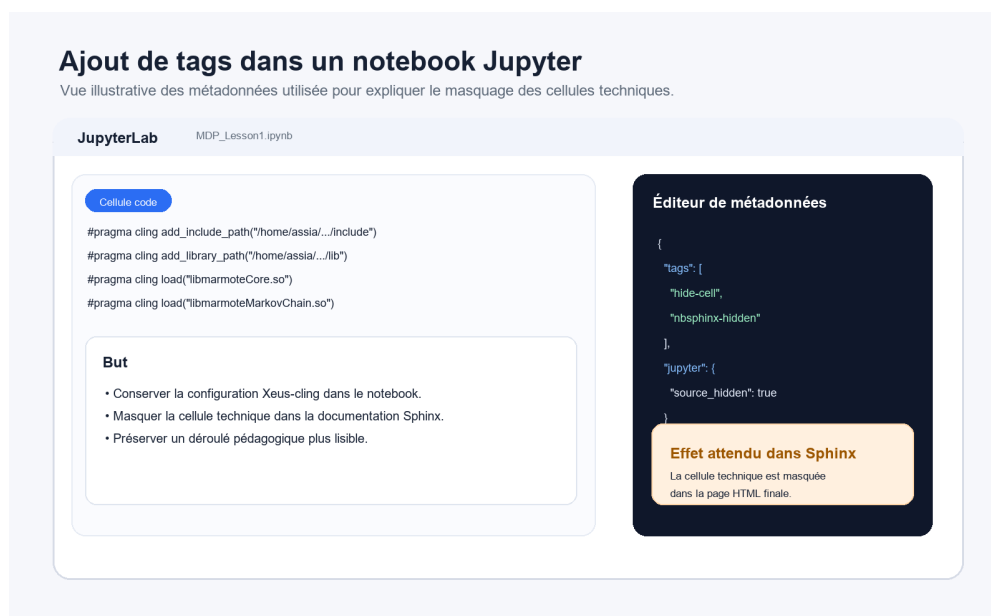


FIGURE 23 – Illustration du masquage des cellules techniques dans les notebooks C++.

Le compromis retenu a été le suivant :

- conserver les cellules `#pragma cling` dans les notebooks pour l'exécution réelle ;
- les taguer comme cellules cachées pour la documentation ;
- garder visibles les `#include` ou les cellules d'initialisation qui ont un intérêt pédagogique ;
- exclure complètement ces cellules cachées lors de la génération des `.cpp` exportés.

Ce choix a permis d'améliorer simultanément :

- l'exécutabilité locale des notebooks ;
- la propreté des pages Sphinx ;
- la qualité des exports C++ distribués.

5.12 Nettoyage final et livrabilité

En fin de projet, une attention particulière a été portée à la livrabilité du dépôt. Cela a concerné :

- le nettoyage et l'harmonisation des commentaires dans les scripts Python actifs ;
- la mise en anglais des docstrings et commentaires techniques lorsque nécessaire ;
- la vérification de la validité syntaxique des scripts de génération ;
- la cohérence entre les dossiers `pytutos`, `cpptutos`, `media` et `instructions`.

Cette phase est moins visible qu'une nouvelle fonctionnalité, mais elle est indispensable dans un contexte de remise, car elle garantit que le dépôt peut être repris par d'autres personnes sans dépendre d'un contexte implicite trop important.

6 Difficultés rencontrées

Même si les réalisations décrites dans les sections précédentes ont permis de faire progresser le projet de manière significative, elles ont été accompagnées de nombreuses difficultés. Ces difficultés n'ont pas toutes eu la même nature : certaines relevaient du contexte d'exécution, d'autres de la documentation existante, d'autres encore des environnements ou des limitations propres aux outils utilisés.

6.1 Difficultés liées à l'arborescence et aux scripts

Les premières difficultés rencontrées relevaient de la structure du projet elle-même. Plusieurs scripts supposaient l'existence de chemins ou de répertoires qui n'étaient pas toujours présents lors des premiers tests. Il a donc fallu reconstituer une arborescence cohérente et réaligner certains mappings entre exemples, exécutable et destinations.

Une autre difficulté importante concernait la sensibilité des scripts au répertoire de lancement. Sans contrôle strict du contexte d'exécution, les copies et déplacements de fichiers devenaient rapidement source d'erreurs. Cela a justifié l'ajout de la redirection automatique et du mode `dry_run`.

6.2 Difficultés liées aux environnements

La préparation des environnements a également constitué un point sensible. En particulier, l'intégration de Xeus-cling s'est heurtée à une différence nette entre Linux et Windows. Sous Windows, l'absence de packages prêts à l'emploi rend l'installation plus fragile et expose davantage à des problèmes de toolchain et de compatibilité.

Les difficultés principales ont été les suivantes :

- dépendances manquantes dans certains environnements ;
- nécessité d'empiler plusieurs environnements Conda selon les besoins ;
- disponibilité inégale des kernels entre les environnements de développement et de build ;
- nécessité de conserver une cohérence entre Jupyter, Xeus-cling et Marmote.

6.3 Difficultés liées aux notebooks C++

La partie la plus délicate du projet a sans doute été l'intégration des notebooks C++. Plusieurs problèmes sont apparus de manière récurrente :

- kernel qui meurt lorsqu'une cellule appelle du code non résolu ;
- ambiguïtés de surcharge propres à Cling ;
- différences entre les fonctions documentées et celles réellement exploitables ;
- réexécution de cellules laissant des objets ou pointeurs dans un état invalide ;
- difficulté à obtenir des sorties à la fois informatives et pédagogiques.

À cela s'ajoute une difficulté plus méthodologique : un notebook n'est pas un exécutable linéaire classique. Il faut donc penser non seulement au code correct, mais aussi à l'ordre d'exécution des cellules, à la stabilité du kernel et à la lisibilité des sorties.

6.4 Difficultés liées à la publication documentaire

La génération Sphinx a elle aussi révélé plusieurs fragilités :

- extension `recommonmark` absente ;
- `pandoc` manquant pour le rendu des notebooks ;

- kernel `xcpp17` non trouvé dans certains contextes de build ;
- liens de téléchargement pointant vers des fichiers absents ;
- incohérences entre les anciens index C++ et la nouvelle organisation en `cpptutos`.

Ces difficultés ont montré que la production documentaire ne dépend pas seulement du contenu des notebooks, mais d’une chaîne plus large : index RST, dépendances Sphinx, kernels disponibles, archives générées et cohérence des chemins publiés.

6.5 Bilan des difficultés

Dans l’ensemble, les difficultés rencontrées ont confirmé qu’il ne s’agissait pas d’un simple travail de conversion de fichiers, mais bien d’un travail de maintenance évolutive sur un workflow complet. La principale leçon méthodologique est qu’il faut traiter ensemble :

- la structure du dépôt ;
- les scripts de génération ;
- les environnements d’exécution ;
- le contenu pédagogique ;
- et la publication finale.

Isoler un seul de ces aspects aurait conduit à une solution partielle. C’est précisément parce que les problèmes ont été abordés de manière globale qu’il a été possible d’aboutir à un résultat cohérent.

7 Conclusion

Ce projet m’a permis de travailler sur un cas concret d’évolution d’un workflow de documentation scientifique, à l’intersection de plusieurs domaines : Python, C++, notebooks Jupyter, génération Sphinx, environnements Conda et maintenance de scripts.

Le travail réalisé ne s’est pas limité à l’ajout de notebooks C++. Il a aussi consisté à :

- stabiliser un workflow existant ;
- rendre les scripts plus robustes et plus sûrs ;
- rapprocher les contenus Python et C++ ;
- produire des exports C++ propres à partir des notebooks ;
- améliorer la qualité des pages et archives diffusées dans la documentation.

Au-delà des résultats techniques, ce projet m’a appris à travailler dans une logique de maintenance progressive : comprendre l’existant, identifier les dépendances réelles, corriger sans casser, puis étendre le système de manière cohérente. Il m’a également montré qu’en documentation scientifique, la fidélité entre le texte, les exemples et l’implémentation réelle est un enjeu majeur.

Les résultats obtenus constituent une base plus solide pour la suite du projet Marmote. Les notebooks C++ sont désormais intégrés dans une logique documentaire cohérente, les archives sont mieux maîtrisées, et le workflow global est plus lisible et plus industrialisable.

Ce projet m’a permis de renforcer mes compétences en ingénierie logicielle, en documentation scientifique et en gestion d’environnements complexes, compétences que je souhaite continuer à développer dans mes futurs projets.

A Annexes

A.1 Installation de Sphinx et des extensions

```
1 python -m pip install -U sphinx sphinx-rtd-theme myst-parser nbsphinx
```

Listing 1 – Installation de Sphinx et des extensions utiles.

A.2 Génération de la documentation HTML avec Sphinx

```
1 python -m sphinx -b html source build/html
```

Listing 2 – Commande de build Sphinx depuis doc/html.

A.3 Création de l'environnement Xeus-cling

```
1 conda create -n xeus-cpp-env -c conda-forge python=3.12 jupyterlab  
   xeus-cling -y  
2 conda activate xeus-cpp-env  
3 conda install -c marmote -c conda-forge marmote -y
```

Listing 3 – Création d'un environnement Conda pour les notebooks C++.

A.4 Empilement d'environnements Conda

```
1 conda activate marmote-use  
2 conda activate --stack xeus-cpp-env
```

Listing 4 – Empilement d'environnements pour combiner outils Marmote et kernel C++.

A.5 Vérification des kernels Jupyter

```
1 jupyter kernelspec list
```

Listing 5 – Exemple de liste des kernels disponibles.

A.6 Extrait de redirection automatique vers le bon dossier

```
1 current_directory = os.getcwd()  
2  
3 if "project-root" in current_directory:  
4     if os.path.basename(current_directory) == "source":  
5         print("No directory change needed", os.getcwd())  
6     else:  
7         print("Unexpected working directory. Redirecting to the  
correct location...")  
8         while os.path.basename(os.getcwd()) != "project-root":  
9             print("Moving up...", os.getcwd())  
10            os.chdir("..")  
11            os.chdir(os.path.join("doc", "html", "source"))  
12            print("Redirection completed successfully:", os.getcwd())  
13 else:
```

```

14     print("Wrong directory. Program stops")
15     sys.exit(1)

```

Listing 6 – Principe de redirection vers doc/html/source.

A.7 Extrait du parsing d'arguments

```

1 def parse_arguments():
2     parser = argparse.ArgumentParser()
3     parser.add_argument("-n", "--dry_run", action="store_true")
4     parser.add_argument("-g", "--generate_only", action="store_true")
5     return parser.parse_args()

```

Listing 7 – Arguments globaux utilisés par total.py.

A.8 Extrait de génération des exports C++

```

1 exporter = ScriptExporter()
2 body, _ = exporter.from_notebook_node(filtered_notebook)
3
4 # Hidden technical cells are excluded before export.
5 # Cling-specific lines such as '#pragma cling ...' are stripped
6 # from the generated .cpp files.

```

Listing 8 – Principe d'export des notebooks C++ vers des scripts.

A.9 Exemple de tags pour masquer une cellule technique

```

1 {
2     "tags": [
3         "hide-cell",
4         "nbsphinx-hidden"
5     ],
6     "jupyter": {
7         "source_hidden": true
8     }
9 }

```

Listing 9 – Métadonnées utilisées pour masquer une cellule dans la documentation.

A.10 Configuration Marmote dans un notebook C++

```

1 #pragma cling add_include_path("/home/assia/miniconda3/envs/xeus-cpp-
   env/include")
2 #pragma cling add_include_path("/home/assia/miniconda3/envs/xeus-cpp-
   env/include/marmoteCore")
3 #pragma cling add_include_path("/home/assia/miniconda3/envs/xeus-cpp-
   env/include/marmoteMarkovChain")
4 #pragma cling add_library_path("/home/assia/miniconda3/envs/xeus-cpp-
   env/lib")
5 #pragma cling load("libmarmoteCore.so")
6 #pragma cling load("libmarmoteMarkovChain.so")

```

Listing 10 – Configuration Xeus-cling et Marmote dans un notebook C++.

A.11 Vérification de l'installation de Marmote dans Colab

```
1 !conda list marmote
2
3 import sys
4 site_packages_path = '/usr/local/lib/python3.11/site-packages'
5 if site_packages_path not in sys.path:
6     sys.path.insert(0, site_packages_path)
7 print(f"sys.path updated: {site_packages_path in sys.path}")
```

Listing 11 – Cellules ajoutées pour vérifier Marmote dans un notebook Colab.